PatchCore Local Running Report on MVTec AD

Xiaokun Wei

July 8, 2026

Abstract

This report summarizes a local full-category PatchCore experiment on the MVTec AD dataset. The experiment was conducted on a Windows Conda environment with an NVIDIA GeForce RTX 4060 Laptop GPU. The goal of this run was to verify whether the official PatchCore pipeline can be executed locally, collect training and inference resource statistics, evaluate category-level anomaly detection performance, and generate visualization examples for qualitative inspection.

1 Report Overview

This report covers the following parts:

1. Local environment, code source, and running command.
2. Explanation of loss function .
3. Output files, saved models, and visualization results.
4. Training time, inference speed, GPU memory usage, and model size.
5. Category-level AUROC results.
6. Visualization examples comparing original images, ground-truth masks, and PatchCore anomaly heatmaps.

2 Experimental Environment and Running Setup

The local running environment is summarized in Table 1.

Table 1: Experimental environment and running configuration.

Item	Value
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
CUDA available	True
PyTorch version	2.4.1+cu124
CUDA version	12.4
Dataset	MVTec AD, 15 categories
Backbone	WideResNet50 pretrained on ImageNet
Image preprocessing	Resize 256, crop 224
Coreset ratio	0.01
Batch size	2
FAISS backend	CPU
Operating system	Windows with Conda environment

The main running command used in this experiment is shown below:

```
python tools\run_patchcore_benchmark.py --category all --gpu 0
--coreset 0.01 --resize 256 --imagesize 224 --batch-si 2
--num-workers 0 --max-visuals 8
```

3 Loss

In this experiment, no traditional training loss curve was generated because PatchCore does not optimize a deep neural network using a conventional loss function. The main computational steps are feature extraction, memory bank construction, coreset sampling, and nearest-neighbor search.

4 Key Results

The overall results are summarized in Table 2.

Table 2: Key results of the full-category PatchCore run.

Metric	Value
Total running time	1076.076 seconds, approximately 17.93 minutes
Total saved model size	111.11 MiB
Average Image AUROC	0.9896
Average Full Pixel AUROC	0.9794
Average Anomaly Pixel AUROC	0.9718
Number of categories	15
Visualization images per category	8

Overall, the local PatchCore run achieved high image-level and pixel-level AUROC scores on MVTec AD. The average Image AUROC reached 0.9896, showing that the method performs well for category-level anomaly detection under the current configuration. The pixel-level results are also strong, with an average Full Pixel AUROC of 0.9794 and an average Anomaly Pixel AUROC of 0.9718.

5 Category-Level Detection Metrics

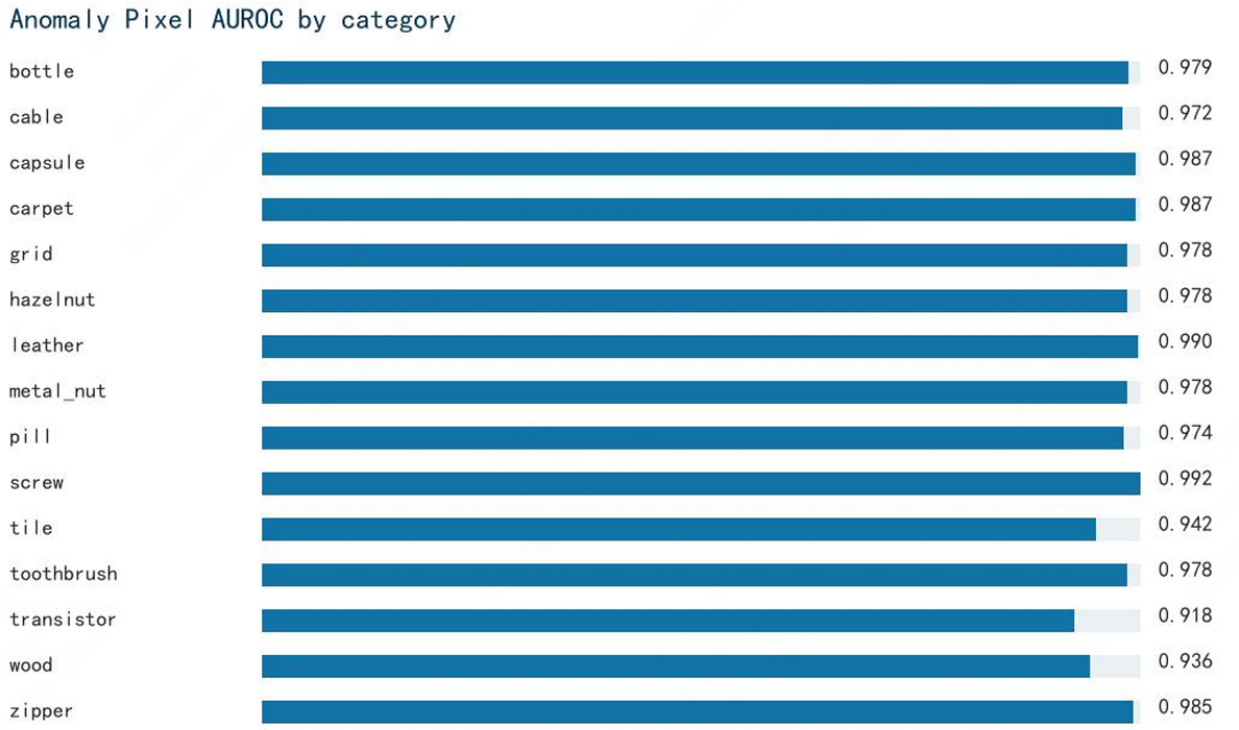
Table 3 shows the category-level Image AUROC, Full Pixel AUROC, and Anomaly Pixel AUROC.

Table 3: Category-level PatchCore detection metrics on MVTec AD.

Category	Image AUROC	Full Pixel AUROC	Anomaly Pixel AUROC
bottle	1.0000	0.9845	0.9791
cable	0.9903	0.9811	0.9720
capsule	0.9701	0.9895	0.9874
carpet	0.9856	0.9901	0.9875
grid	0.9866	0.9831	0.9784
hazelnut	1.0000	0.9858	0.9776
leather	1.0000	0.9929	0.9904
metal_nut	0.9971	0.9826	0.9784
pill	0.9793	0.9757	0.9738
screw	0.9615	0.9937	0.9923
tile	0.9913	0.9585	0.9422
toothbrush	0.9972	0.9845	0.9784
transistor	0.9950	0.9519	0.9184
wood	0.9947	0.9496	0.9358
zipper	0.9958	0.9882	0.9850

Several categories achieved perfect or near-perfect Image AUROC, such as bottle, hazelnut, and leather. The lowest Image AUROC was observed on screw, with a value of 0.9615, but it still remained at a high level. For pixel-level localization, transistor and wood had relatively

lower Anomaly Pixel AUROC compared with other categories, which suggests that their defect localization may be more challenging.



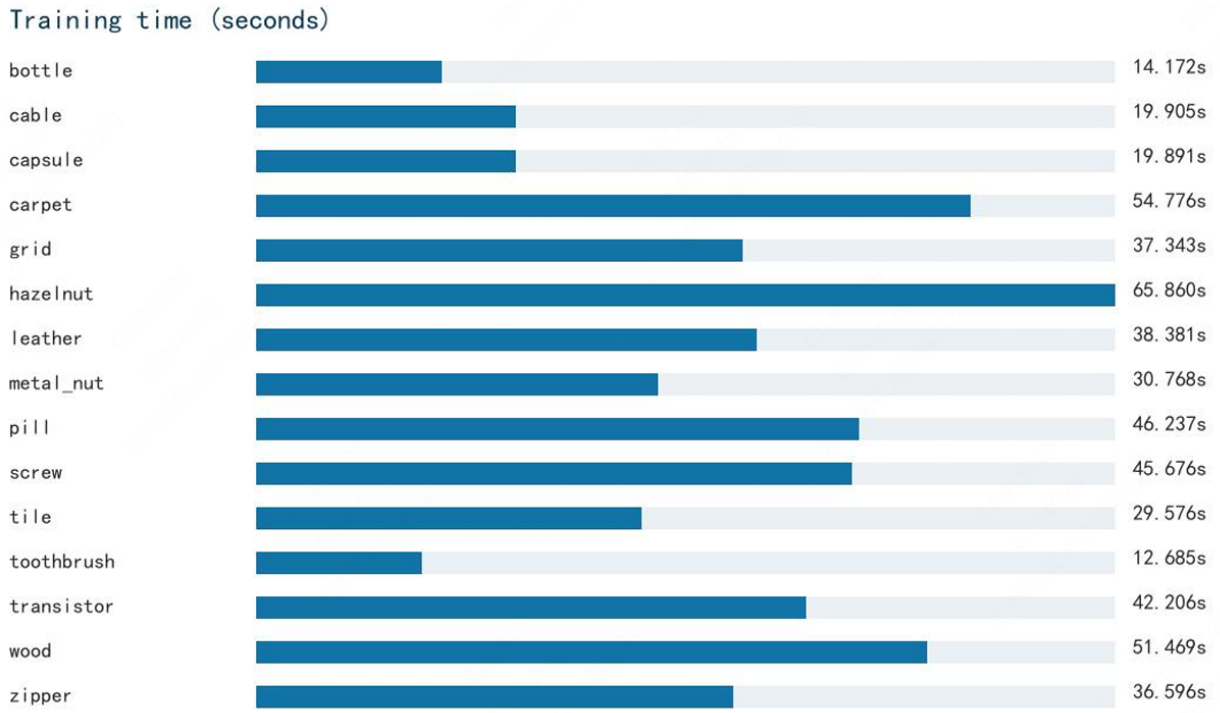
6 Training Resources and Time

Table 4 summarizes the number of training images, training time, and peak CUDA memory usage for each category.

Table 4: Training time and peak GPU memory usage by category.

Category	Training Images	Training Time (s)	Peak CUDA Memory (MiB)
bottle	209	14.172	1022.430
cable	224	19.905	1079.120
capsule	219	19.891	1059.440
carpet	280	54.776	1280.550
grid	264	37.343	1224.330
hazelnut	391	65.860	1675.220
leather	245	38.381	1150.070
metal_nut	220	30.768	1065.800
pill	267	46.237	1228.970
screw	320	45.676	1421.710
tile	230	29.576	1101.020
toothbrush	60	12.685	493.570
transistor	213	42.206	1036.200
wood	247	51.469	1160.150
zipper	240	36.596	1139.750

The highest training time was observed for hazelnut, which took 65.860 seconds. This category also had the largest number of training images and the highest peak CUDA memory usage, reaching 1675.220 MiB. The toothbrush category was the lightest in this run, with only 60 training images, 12.685 seconds of training time, and 493.570 MiB of peak CUDA memory.



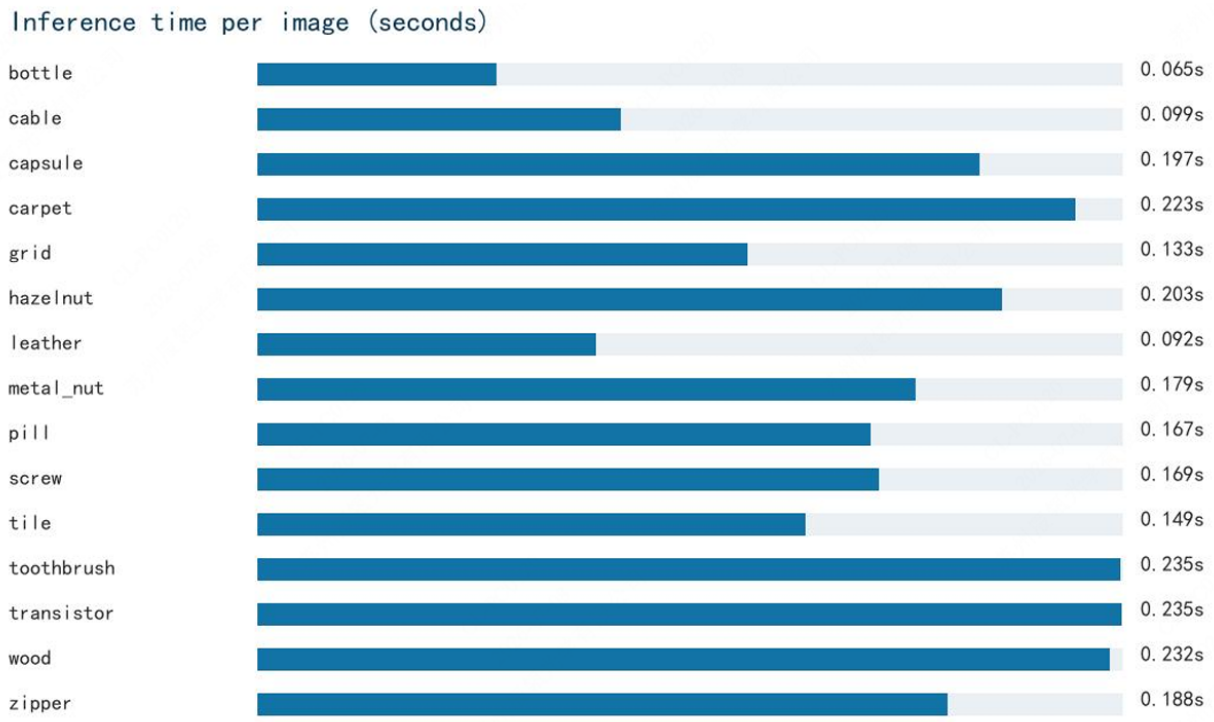
7 Inference Speed

Table 5 shows the inference time and average inference time per test image.

Table 5: Inference speed by category.

Category	Test Images	Inference Time (s)	Seconds per Image
bottle	83	5.406	0.0651
cable	150	14.811	0.0987
capsule	132	25.961	0.1967
carpet	117	26.058	0.2227
grid	78	10.396	0.1333
hazelnut	110	22.279	0.2025
leather	124	11.436	0.0922
metal_nut	115	20.586	0.1790
pill	167	27.857	0.1668
screw	160	27.064	0.1691
tile	117	17.460	0.1492
toothbrush	42	9.874	0.2351
transistor	100	23.543	0.2354
wood	79	18.332	0.2321
zipper	151	28.373	0.1879

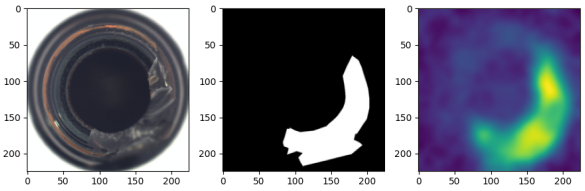
The fastest inference speed was observed for bottle, with an average of 0.0651 seconds per image. The slowest categories were transistor, toothbrush, and wood, each taking more than 0.23 seconds per image. The inference speed is acceptable for local experimentation, although further optimization may be needed for real-time industrial deployment.



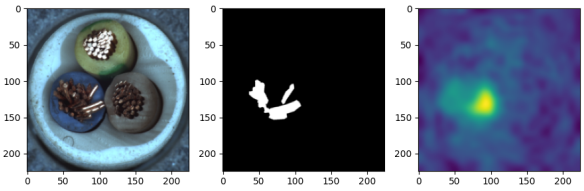
8 Inference Speed

9 Visualization Examples

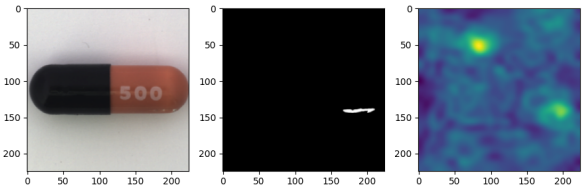
It also generated side-by-side visualization images for qualitative inspection. Each visualization compares the original image, the ground-truth mask, and the PatchCore anomaly heatmap. For each category, up to 8 visualization samples were saved.And here are parts of the results:



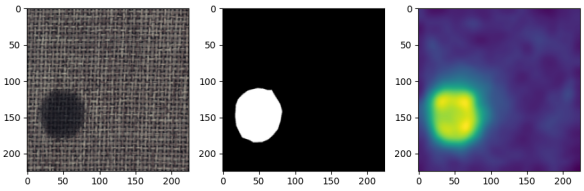
Bottle: broken large



Cable: bent wire



Capsule: crack



Carpet: color defect

Figure 1: Qualitative PatchCore visualization examples from four MVTec AD categories. Each sample compares the original image, the ground-truth mask, and the predicted anomaly heatmap.

10 Conclusion

This experiment successfully verified that the official PatchCore code can be locally executed on a Windows Conda environment with an NVIDIA RTX 4060 Laptop GPU. All 15 categories of the MVTec AD dataset were completed in a full-category benchmark run.

The current configuration achieved high detection performance, with an average Image AUROC of 0.9896, an average Full Pixel AUROC of 0.9794, and an average Anomaly Pixel AUROC of 0.9718. The training time, inference speed, GPU memory usage, and model size were all acceptable for local experimentation.

References

[1] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards Total Recall in Industrial Anomaly Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[2] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1038–1059, 2021.